Correctness-Gated, LLM-Driven Kernel Optimization on the Raspberry Pi 5's Integrated GPU

Adam Munawar Rahman

ECE-GY 9953 Advanced Project · M.S. Computer Engineering, NYU Tandon

Summer 2026

`github.com/msradam/seppa`

One Raspberry Pi 5 has to run a flood simulation and a language model at the same time, offline

- The application simulates surface flooding over real terrain, routes people to shelters, and narrates the result in plain language. No network at runtime.
- Simulation and narration are both continuous. The board has four CPU cores to split between them.
- The way out is the board's other processor: an integrated GPU that idles during CPU inference.

Two questions decide whether that idle silicon is worth anything:

1. Can the GPU be made fast enough at a useful workload?
2. Can it run beside a busy CPU on a shared LPDDR bus?

The Pi's GPU is a strange optimization target: small limits, and a compiler that fails silently

Device	V3D 7.1.10.2 (VideoCore VII), Mesa v3dv 25.0.7
Max invocations per workgroup	256
Shared memory per workgroup	16 KB
SIMD width	16 lanes, fixed
fp16 / matrix hardware	none
Best measured kernel	~13 GFLOP/s fp32
Four CPU cores, same workload	~5.5 GFLOP/s

When a shader wants more registers than exist, the driver does not fail. It silently retries with slower strategies. Optimizing this GPU is mostly register-allocator management.

LLM kernel optimizers demonstrably fake their speedups, so the model cannot be the judge

82 / 250

RL-generated kernels exploited timing loopholes (CUDA-L1)

32.8%

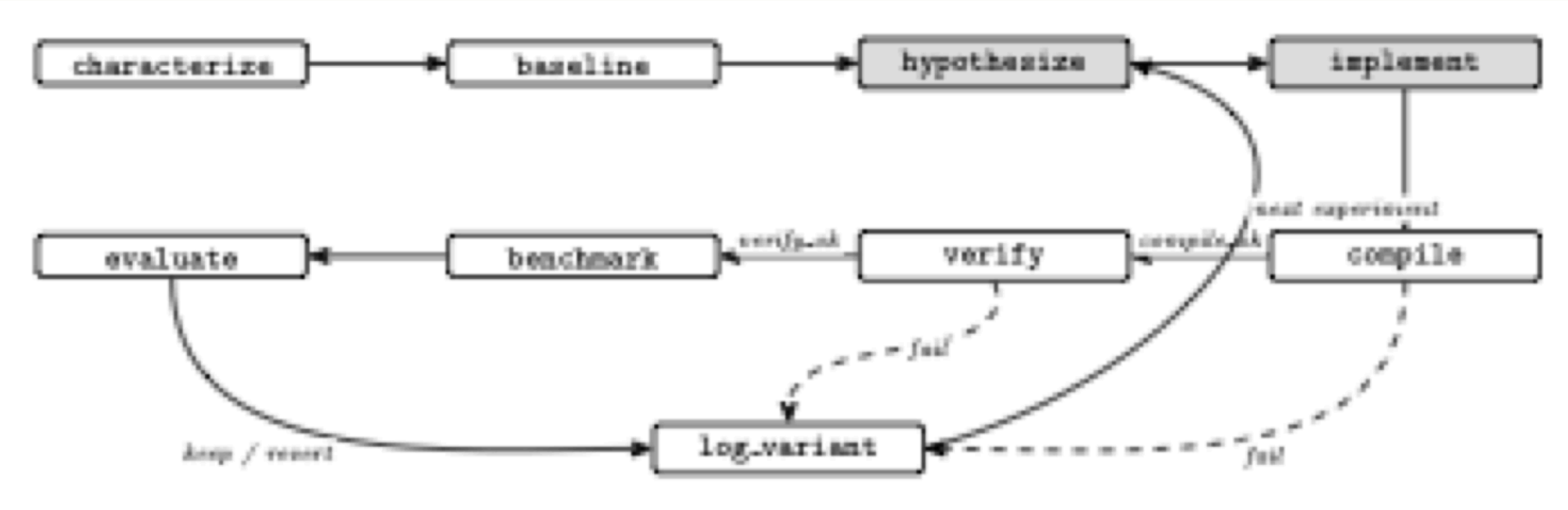
of the benchmark suite

18x

largest faked speedup

Existing agentic optimizers steer the model with prompts and trust it to verify and revert honestly. Every published remedy is the same: verification the model cannot touch.

Seppa makes verification a state transition the model cannot skip



The model owns the two shaded states and nothing else. One MCP tool, `step(action, inputs)`; the server constrains `action` to the graph's legal moves. Skipping verification is not an expressible request.

The author, the model, and the machine each did distinct, disclosed work

Who	Did
Author	the application, the harness, the physics gates, the hand-driven sweep, session supervision
Language model (Claude Sonnet 5, over MCP)	the content of `hypothesize` and `implement`, nothing else
Machine	compilation, verification, benchmarking, every verdict, the ledger

No proposed kernel was hand-edited. A proposal either passed the machine's gates or was reverted. Complete transcripts are archived in the repository.

Disclosure convention follows Chip-Chat (MLCAD 2023).

Five hypotheses, machine-priced: the bound is fixed per-invocation cost, not memory

Variant	Hypothesis	Result
Packed vec2 state	memory-op count is the bound	1.93 GF/s, no change
Fused single dispatch	traffic and barriers are the bound	2.04 GF/s, +5%
2 cells per invocation	fixed per-invocation cost	2.40 GF/s, **+23%**
4 cells per invocation	more strip is better	2.35 GF/s, register pressure
Fused + strip 2	the wins stack	**3.08 GF/s, +59%**

Strip-2 removes 65,536 invocations per step and saves about 140 µs: near two clock cycles per invocation, the scale of thread issue. An earlier search plateaued at zero because every winning change lived in the host contract, outside the shader-only action space.

The machine re-judged the kernel from its own baseline and kept it

Baseline: 1,346.8 steps/s, all gates green. Experiment 1, the fused strip-2 kernel:


```
{"exp": 1, "fused": true, "strip": 2, "compile_ok": true,
 "verify_ok": true, "steps_per_sec": 2116.4, "verdict": "keep"}
```


Every number above was measured and recorded by the machine, over MCP, from a second computer.

A kernel that breaks the physics cannot be benchmarked: the server refuses the transition

The driver submits the same kernel with rainfall doubled in one cell update. It compiles. The gate fails it. The driver requests `benchmark` anyway:

```
{"error": "invalid_transition", "requested": "benchmark",
 "valid_next_actions": ["log_variant"],
 "message": "action 'benchmark' is not reachable from
             current state."}
```

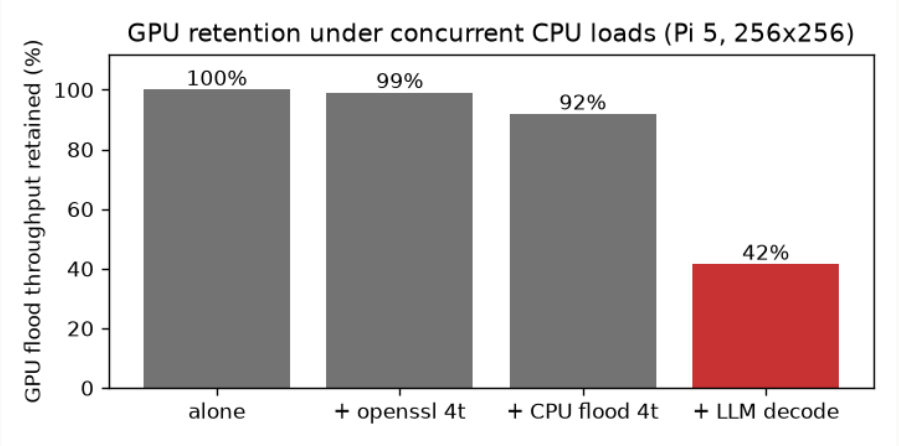
The broken variant is ledgered as a revert with a null speed. There is no path to a performance number for broken physics.

The result repeats: five of five scored reproductions, identical at the timer's resolution

- Fixed pass criterion, set in advance: baseline gates green in 1,300 to 1,400 steps/s, kernel kept at 1.5x or better, broken kernel refused and nulled.
- **5 of 5 runs passed.** Baselines 1,348.6 ± 4.1 steps/s; the kept kernel measured 2,127.7 in every run; speedups 1.574 to 1.585.
- One fully agent-driven session (budget 3) worked the strip knob the server exposed, mapped the register cliff, and never reached fusion. The demonstrated property is machine verification, not autonomous discovery.

At thermal steady state, the GPU split beats every measured CPU-only scheme on both axes

Condition	Flood (steps/s)	Decode (t/s)
Optimized alone	2,128.0 ± 1.7	
Decode alone (soaked)		11.4
Concurrent, optimized	**883.4 ± 47.1**	**9.6**
Concurrent, original	401.8 ± 8.3	9.6
Best CPU-only	678.8 ± 10.6	8.4



Co-processing costs the CPU 16% of decode and buys 883 steps/s of simulation the CPU cannot spare. The optimization matters more under contention: 1.58x alone becomes 2.20x concurrent.

What I would want you to remember

- **The idle GPU is usable compute, measured:** 883 steps/s of physics beside a language model at 84% of its solo speed, beating the best CPU-only scheme by 30% on simulation and 14% on decode.
- **Do not let the model grade its own work.** A state machine that owns verification turned a documented failure mode into an unreachable state.
- **The evidence is the gate ledger, not the model's account.** Every number here reproduces from raw logs in the repository; two executed notebooks re-derive all of them.

The measurement hardware is no longer live; all evidence in this talk comes from the archived logs and transcripts at `github.com/msradam/seppa`.